Compte rendu de TP

Feux de Carrefour

Réalisé par :

- **Mohamed Wahib Elfahem**
- Islem Guesmi
- Fadwa Bougoffa

1.Objectif du TP :

L'objectif de ce TP est de programmer un microcontrôleur STM32F407 en langage C afin de commander un feu tricolore composé de trois LED : rouge, jaune et verte.

Ce TP permet de comprendre la configuration des broches GPIO en mode sortie, l'activation des horloges des périphériques, la manipulation des registres du microcontrôleur et l'utilisation d'une temporisation logicielle pour contrôler la durée d'allumage de chaque LED.

2. Le microcontrôleur STM32F407:

Le STM32F407 est un microcontrôleur de la famille STM32, basé sur le processeur ARM Cortex-M4. Il intègre plusieurs périphériques, notamment des ports GPIO permettant de communiquer avec des composants externes tels que des LED, des boutons-poussoirs et des capteurs numériques.

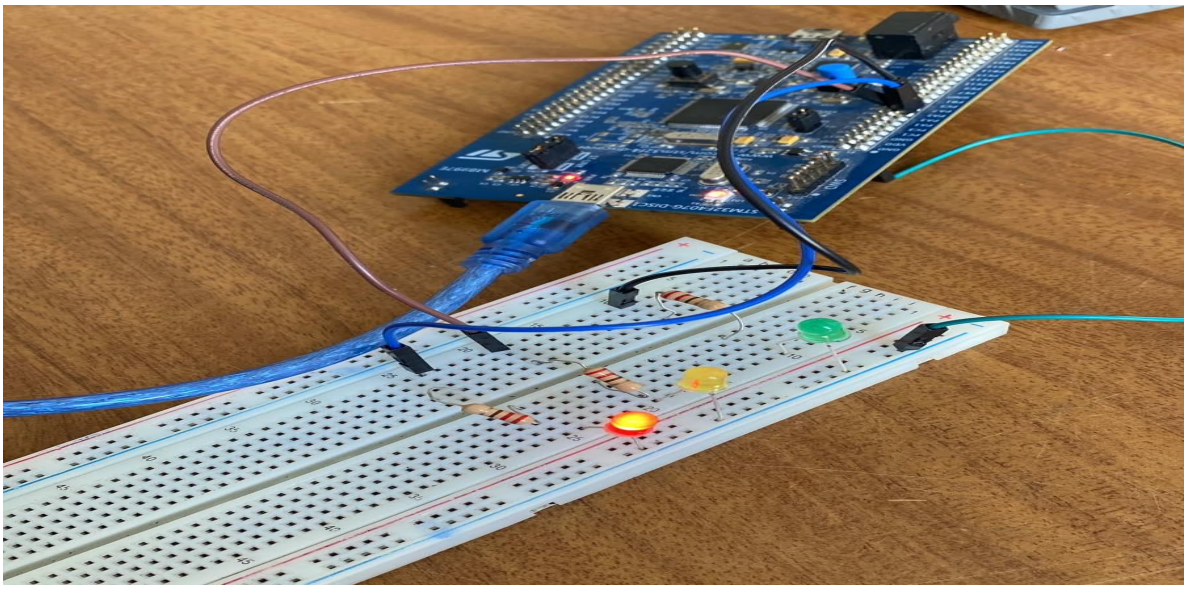
Étape1 : Cablage du hardware :

Le câblage de ce TP consiste à :

réaliser un feu tricolore à l'aide d'un microcontrôleur STM32F407, de trois LED (rouge, jaune et verte), de trois résistances et de fils de connexion :

- **La LED rouge est connectée à la broche PA5.**
- **la LED jaune à la broche PA6 .**
- **la LED verte à la broche PA7 du microcontrôleur.**

- **Chaque LED est associée à une résistance de limitation de courant, généralement de 220 Ω à 330 Ω, afin de protéger la LED et la sortie du microcontrôleur contre un courant excessif.**
- **L'anode de chaque LED est reliée à sa broche de commande à travers une résistance, tandis que sa cathode est connectée à la masse (GND).**
- **Le STM32F407 constitue l'unité de commande du système : il exécute le programme en langage C et contrôle l'allumage et l'extinction des LED grâce aux sorties GPIO.**
- **Les résistances assurent la limitation du courant.**
- **les LED représentent les trois couleurs du feu de signalisation .**
- **les fils permettent d'établir les connexions électriques entre les composants.**
- **Enfin, la fonction de temporisation delay() permet de maintenir chaque LED allumée pendant une durée déterminée avant de passer à la couleur suivante.**



Étape2 : Configuration logicielle STM32CubeMX :



Étape 3 : Écrire le programme en langage C dans Keil µVision5 :

main.c

```c
#include "stm32f407xx.h"

void delay(volatile uint32_t time)
{
    while (time--)
    {
        __NOP();
    }
}

int main(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;

    /* PA5 rouge, PA6 jaune, PA7 vert */
    GPIOA->MODER &= ~(3U << 10);
    GPIOA->MODER &= ~(3U << 12);
    GPIOA->MODER &= ~(3U << 14);

    GPIOA->MODER |= (1U << 10);
    GPIOA->MODER |= (1U << 12);
    GPIOA->MODER |= (1U << 14);

    while (1)
    {
        /* Rouge */
        GPIOA->ODR = (1U << 5);
        delay(9000000);

        /* Jaune  */
        GPIOA->ODR = (1U << 6);
        delay(4000000);

        /* Vert */
        GPIOA->ODR = (1U << 7);
        delay(9000000);

        /* Jaune */
        GPIOA->ODR = (1U << 6);
        delay(4000000);
    }
}
```

3.Explication du Programme :

```
#include "stm32f407xx.h"  // Bibliothèque du microcontrôleur STM32F407
// Fonction de temporisation logicielle
void delay(volatile uint32_t time)
{
// Répéter la boucle jusqu'à ce que time atteigne zéro
while (time--)
{
__NOP();  // Instruction vide qui consomme quelques cycles processeur
}
}
int main(void)
{
// Activer l'horloge du port GPIOA
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
// Configurer PA5, PA6 et PA7 en mode sortie
// Chaque broche utilise deux bits dans le registre MODER
// Mettre à zéro les bits de configuration de PA5
GPIOA->MODER &= ~(3U << 10);
// Mettre à zéro les bits de configuration de PA6
GPIOA->MODER &= ~(3U << 12);
// Mettre à zéro les bits de configuration de PA7
GPIOA->MODER &= ~(3U << 14);
// Configurer PA5 en sortie (01)
GPIOA->MODER |= (1U << 10);
// Configurer PA6 en sortie (01)
```

```
GPIOA->MODER |= (1U << 12);
// Configurer PA7 en sortie (01)
GPIOA->MODER |= (1U << 14);
// Boucle infinie : répéter le cycle du feu tricolore
while (1)
{
// Allumer la LED rouge sur PA5
// Éteindre les autres sorties du port GPIOA
GPIOA->ODR = (1U << 5);
// Attendre pendant 9 000 000 itérations
delay(9000000);
// Allumer la LED jaune sur PA6
// Éteindre les LED rouge et verte
GPIOA->ODR = (1U << 6);
// Attendre pendant 4 000 000 itérations
delay(4000000);
// Allumer la LED verte sur PA7
// Éteindre les LED rouge et jaune
GPIOA->ODR = (1U << 7);
// Attendre pendant 9 000 000 itérations
delay(9000000);
// Allumer de nouveau la LED jaune sur PA6
GPIOA->ODR = (1U << 6);
// Attendre pendant 4 000 000 itérations
delay(4000000);
// Retour au début de la boucle : LED rouge
```

```
}

}
```

4. Résultats obtenus :

- les trois LED s'allument successivement selon une séquence définie.
- La LED rouge, connectée à la broche PA5, s'allume en premier pendant une temporisation de 9 000 000 itérations.
- Ensuite, la LED jaune, connectée à PA6, s'allume pendant 4 000 000 itérations,
- Puis la LED verte, connectée à PA7, pendant 9 000 000 itérations.
- Enfin, la LED jaune s'allume de nouveau pendant 4 000 000 itérations avant que le cycle ne recommence.
- À chaque changement de couleur, la LED précédente s'éteint automatiquement grâce à l'écriture dans le registre ODR.

5. Conclusion :

Ce TP a donc permis de vérifier le fonctionnement des sorties GPIO, la commande des LED et l'utilisation d'une temporisation logicielle pour simuler le fonctionnement d'un feu de carrefour .

! Les durées indiquées correspondent à des itérations et non à des secondes exactes.